

COSC 39: Theory of Computation

Problem 1. Parsing regular expressions.

Can the task of parsing regular expressions itself be implemented using a regular expression? This property is known as universality—the language is powerful enough to recognize its own structure.

We can treat the collection of regular expressions itself as a language RegExp , containing words that are valid regular expressions over the binary alphabet $\{0, 1\}$. To write down a regular expression in RegExp , in addition to 0 and 1 we need a few extra symbols; so we define the alphabet set for RegExp to be

$$\Sigma' := \{0, 1, +, *, (,)\}.$$

- (a) Prove that the language RegExp itself is not regular. In other words, we cannot construct a parser for valid regular expressions using only DFAs/NFAs.
- (b) What if we disallow nested parentheses? (In other words, inside a pair of parentheses (and), no further parentheses can appear.) Prove or disprove that the language RegExp with no-nesting constraint is regular.

Solution.

- (a) Define the fooling set as $F := \{0^m 0 \mid m \in \mathbb{N}\}$. Then for two arbitrary prefixes $0^i 0, 0^j 0 \in F$ with $i \neq j$, let the distinguishing suffix be $(+1)^i$ (Note, the red parenthesis in the suffix are for our reference, for instance, $(+1)^2 = +1 + 1$). Then we have,

- $0^j 0 (+1)^i \notin L$, because $i \neq j$ and hence the parentheses aren't balanced.
- $0^i 0 (+1)^i \in L$, since the parentheses are balanced.

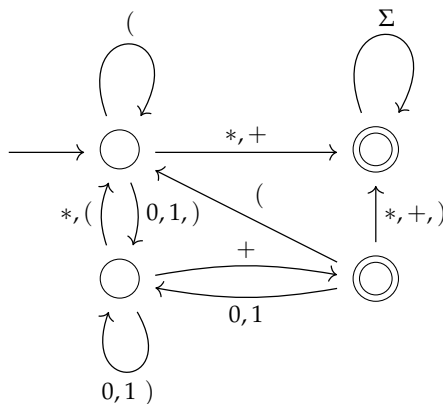
This implies that F is a fooling set of infinite size, and thus L is not regular.

- (b) We prove the language RegExp is regular by construction:

First, construct an NFA which accepts the **complement** of the language RegExp (i.e. the NFA rejects all strings which satisfy the rules of regular expression formatting). We do so by building two distinct *gadgets* to prevent formatting errors.

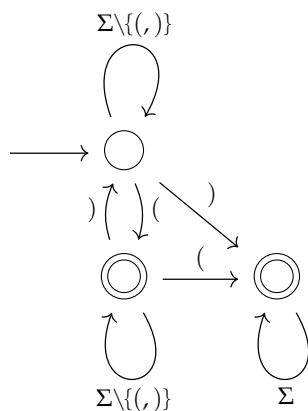
1. (i) $*$ and $+$ characters must follow valid characters, particularly (0, 1, and ')')

- (ii) $+$ characters must be followed by valid characters, particularly $(0, 1,$ and $''('')$



The first gadget ensures proper use of $*$ and $+$ operations:

2. The second gadget guarantees correct use of parentheses—every open parentheses has a corresponding closed parentheses and in the right order—and prevents nesting.



Problem 2. Regular or not?.

Prove or disprove that each of the languages below is regular (or not). Let Σ^+ denote the set of all nonempty strings over alphabet Σ ; in other words,

$$\Sigma^+ = \Sigma \cdot \Sigma^*.$$

(a) $\{wxw^R : w, x \in \Sigma^+\}$

(b) $\{ww^Rx : w, x \in \Sigma^+\}$

Hint: To prove that a language L is regular, construct an NFA that recognizes L ; to disprove that L is regular, construct a fooling set and argue that the construction is correct.

Solution.

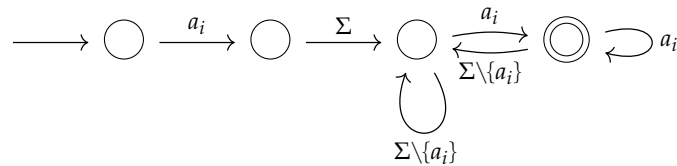
- (a) Let $a_i \in \Sigma$. (Note that this diagram shows the i -th branch of the DFA, for each $a_i \in \Sigma$ there is a separate branch of the DFA with a_i replaced with the character.) The intuition for the following DFA to recognize all strings in

$$\{wxw^R : w, x \in \Sigma^+\}.$$

by satisfying 2 conditions:

1. Since both w and x are in Σ^+ , they must each contain at least one character. Therefore, any string of the form wxw^R has length at least 3.
2. The format wxw^R means that the string begins and ends with the same character, because the first character comes from w , and the last character comes from w^R , which mirrors w .

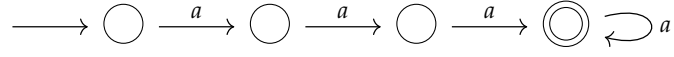
The DFA visualizes this by branching on each possible starting symbol a_i and then ensures that the string ends with the same symbol. The middle portion x can be any sequence of symbols from Σ , as long as it is nonempty. The DFA allows any characters in between while enforcing matching first and last characters.



- (b) When $|\Sigma| = 1$, say $\Sigma = \{a\}$, every $w \in \Sigma^+$ satisfies $w = w^R$. Thus the language becomes:

$$(\Sigma^+ \Sigma^+ \Sigma^+) = \{a^n : n \geq 3\}$$

which is just any string of the alphabet with more than 3 characters. This can be recognized by the DFA below, proving its regular.



Else if $|\Sigma| \geq 2$, we claim that this language is not regular. Let $0, 1$ be two arbitrary characters in Σ . Define the fooling set as $F := \{(01)^m \mid m \in \mathbb{N}\}$. Then for two arbitrary prefixes $(01)^i, (01)^j \in F$ with $i > j$, let the distinguishing suffix be $(10)^i$. Then we have,

- $(01)^i(10)^i \notin L$, because any valid decomposition must have $w = (01)^k$, so $w^R = (10)^k$. This forces $k = i$, which leaves $x = \varepsilon$. But since x must be non-empty, this is impossible.
- $(01)^j(10)^i \in L$, with $w = (01)^j$, $w^R = (10)^j$, $x = (10)^{i-j}$.

This implies that F is a fooling set of infinite size, and thus L is not regular.